

Compte-Rendu de Projet

Ultimate Tic Tac Toe

IA Minimax avec élagage Alpha-Beta

Membres du groupe

GOUDON Octave GOUTODIER Lucas GOUEL Julien

7 mai 2026

Table des matières

1	Introduction	3
2	Représentation de l'état du jeu	3
2.1	Structure de données	3
2.2	Immutabilité de l'état	3
3	Logique du jeu	4
3.1	Coups valides	4
3.2	Application d'un coup	4
3.3	Détection de fin de partie	4
4	Minimax avec élagage Alpha-Beta	4
4.1	Principe	4
4.2	Élagage Alpha-Beta	5
5	Optimisations empilées	5
5.1	Iterative Deepening (IDDFS)	5
5.2	Transposition Table avec flags (EXACT / LOWER / UPPER)	6
5.3	Hash de Zobrist	6
5.4	Killer Moves	7
5.5	History Heuristic	7
5.6	Aspiration Windows	7
5.7	Late Move Reduction (LMR)	8
5.8	Move Ordering unifié	9
6	Heuristique d'évaluation	9
6.1	Rôle et enjeux	9
6.2	Cinq critères combinés	9
7	Interface et modes de jeu	10
7.1	Affichage texte	10
7.2	Convention de saisie	10
7.3	Mode IA vs IA	11
8	Analyse des performances	11
8.1	Gain de l'Iterative Deepening	11
8.2	Impact de la transposition table	11
8.3	Synergie des optimisations	11
8.4	Limites et pistes d'amélioration	11

9 Conclusion

12

1. Introduction

L’Ultimate Tic Tac Toe est une extension combinatoire du Morpion classique. Le jeu se déroule sur une grille de 9×9 cases, divisée en 9 mini-morpions de 3×3 . L’objectif est de remporter 3 mini-morpions alignés en ligne, colonne ou diagonale dans la macro-grille.

La règle fondamentale qui donne sa profondeur stratégique au jeu est la **contrainte de redirection** : la case jouée dans un mini-morpion détermine le mini-morpion dans lequel l’adversaire devra obligatoirement jouer ensuite. Si ce morpion est déjà terminé ou plein, le joueur adverse est libre de jouer n’importe où. Cette contrainte crée des dépendances complexes entre chaque décision locale et ses conséquences globales.

Ce rapport présente l’architecture de notre IA, les choix algorithmiques effectués, et les sept optimisations empilées sur la base Minimax Alpha-Beta pour maximiser la qualité du jeu dans la limite de 10 secondes par coup.

2. Représentation de l’état du jeu

2.1 Structure de données

L’état complet du jeu est un dictionnaire Python à cinq champs :

- `cells` : liste de 9 listes de 9 cases (`None` / `'X'` / `'O'`) — les 9 mini-morpions ;
- `macro_winner` : liste de 9 valeurs (`None` / `'X'` / `'O'` / `'D'`) — résultat de chaque mini-morpion ;
- `active_board` : entier 0–8 ou `None` — mini-morpion dans lequel le prochain coup doit être joué ;
- `turn` : `'X'` ou `'O'` — joueur dont c’est le tour ;
- `zhash` : entier 64 bits — hash de Zobrist de l’état courant (voir section 5.3).

Les cases et mini-morpions sont indexés de 0 à 8 selon le schéma ligne d’abord :

0	1	2
3	4	5
6	7	8

Un coup est identifié par une paire (bi, ci) : bi est l’indice du mini-morpion (0–8), ci est l’indice de la case dans ce morpion (0–8).

2.2 Immutabilité de l’état

L’état n’est jamais modifié en place. La fonction `apply_move()` crée systématiquement une copie via `copy_state()` avant toute modification. Ce choix garantit qu’aucune branche de l’arbre Minimax n’affecte les autres : il n’y a pas de fonction d’annulation (*undo*), ce qui simplifie le code et élimine une source classique de bugs.

Le hash de Zobrist est mis à jour de façon incrémentale à chaque `apply_move()` par un simple XOR, ce qui le rend aussi rapide qu'une affectation entière.

3. Logique du jeu

3.1 Coups valides

La fonction `valid_moves(st)` retourne tous les coups (*bi*, *ci*) jouables. Elle respecte la règle de redirection :

- Si `active_board` est défini : le joueur ne peut jouer que dans ce mini-morpion ;
- Si `active_board` est `None` : le joueur peut jouer dans tout morpion non terminé et non plein.

3.2 Application d'un coup

`apply_move(st, bi, ci)` effectue dans l'ordre : placement du pion, mise à jour incrémentale du hash de Zobrist (XOR sur la case placée et sur le bit de tour), vérification du résultat du mini-morpion (victoire ou match nul), puis calcul du prochain `active_board` selon la règle de redirection :

```
1 next_bi = ci    # la case jouée détermine le prochain morpion
2 if macro_winner[next_bi] is EMPTY and not is_full(cells,
   next_bi):
3     active_board = next_bi
4 else:
5     active_board = EMPTY    # morpion terminé -> libre
```

Listing 1 – Calcul du prochain plateau actif

3.3 Détection de fin de partie

La partie est terminée si un joueur a aligné 3 mini-morpions (vérification sur `macro_winner`), ou s'il n'existe plus de coup valide. En cas de grille pleine sans alignement, le joueur ayant remporté le plus de mini-morpions gagne la partie.

4. Minimax avec élagage Alpha-Beta

4.1 Principe

Le Minimax est un algorithme de recherche récursive qui alterne deux rôles à chaque niveau de profondeur : le nœud *Maximizing* (tour de l'IA, cherche le score maximum) et le nœud *Minimizing* (tour de l'adversaire, cherche le score minimum du point de vue de

l'IA). En remontant les valeurs des feuilles, l'IA choisit le coup garantissant le meilleur score dans le pire cas.

4.2 Élagage Alpha-Beta

L'élagage Alpha-Beta maintient deux bornes à chaque nœud :

- **alpha** : meilleur score garanti pour l'IA (initialisé à $-\infty$);
- **beta** : meilleur score garanti pour l'adversaire (initialisé à $+\infty$).

Dès que $\beta \leq \alpha$, la branche est élaguée : l'adversaire ne laissera jamais l'IA atteindre ce nœud. Dans le meilleur cas (coups triés optimalement), cet élagage réduit la complexité de $O(b^d)$ à $O(b^{d/2})$, ce qui double la profondeur explorable pour un budget de calcul fixe.

```

1 alpha = max(alpha, val)
2 if alpha >= beta:
3     break # coupure beta

```

Listing 2 – Coupure bêta dans le nœud Maximizing

5. Optimisations empilées

Sept optimisations sont combinées sur la base Minimax Alpha-Beta. Leur synergie est clé : chacune amplifie l'efficacité des autres.

5.1 Iterative Deepening (IDDFS)

Au lieu d'une profondeur fixe, l'IA explore profondeur 1, puis 2, puis 3, etc., jusqu'à épuisement du temps (limite : 9 secondes sur les 10 autorisées). À chaque interruption par timeout, elle retourne le meilleur coup de la dernière itération complète.

Cette technique offre trois avantages majeurs : elle garantit toujours un coup valide même si le temps est très court, elle adapte automatiquement la profondeur à la complexité de la position (en fin de partie, l'arbre est petit et on peut aller beaucoup plus profond), et elle fournit les résultats de chaque itération pour améliorer le tri des coups de l'itération suivante.

```

1 for depth in range(MIN_DEPTH, depth_cap + 1):
2     if time.time() > deadline - 0.05:
3         break
4     try:
5         # recherche complète à cette profondeur
6         ...
7         best_move_found = best_this # itération complète ->
            mise à jour
8     except TimeoutError:

```

```
9      break      # garder le résultat de l'itération précédente
```

Listing 3 – Squelette IDDFS avec gestion du timeout

5.2 Transposition Table avec flags (EXACT / LOWER / UPPER)

La transposition table est un dictionnaire qui mémorise les états déjà évalués. La clé est le triplet (zhash, depth, maximizing). Chaque entrée stocke un triplet (valeur, flag, meilleur_coup) où le flag indique la nature de la valeur :

- TT_EXACT : valeur exacte — l'état a été évalué complètement ;
- TT_LOWER : borne inférieure — une coupure alpha s'est produite, la vraie valeur est $\geq$ à celle-ci ;
- TT_UPPER : borne supérieure — une coupure beta s'est produite, la vraie valeur est $\leq$ à celle-ci.

Ces flags permettent de réutiliser les entrées de la TT même pour des recherches partielles, ce qui est beaucoup plus efficace qu'une TT naïve. Le meilleur coup stocké permet en outre de l'essayer en priorité lors de l'itération suivante (voir section 5.8).

```
1 if best <= original_alpha:  flag = TT_UPPER
2 elif best >= beta:          flag = TT_LOWER
3 else:                       flag = TT_EXACT
4 tt[key] = (best, flag, best_move_local)
```

Listing 4 – Stockage en TT avec flag approprié

5.3 Hash de Zobrist

La transposition table est indexée par un hash de Zobrist : chaque triplet (bi, ci, joueur) est associé à un nombre aléatoire de 64 bits tiré à l'initialisation (graine fixe 42 pour la reproductibilité). Un bit de tour est également inclus via ZOBRIST_TURN. Le hash d'un état est le XOR de tous ces nombres pour les cases occupées. La mise à jour est $O(1)$ par coup (deux XOR : case + changement de tour), contrairement à un hash par sérialisation qui serait $O(n)$.

```
1 # Initialisation (une seule fois, seed fixe)
2 random.seed(42)
3 ZOBRIST = {
4     (bi, ci, player): random.getrandbits(64)
5     for bi in range(9)
6     for ci in range(9)
7     for player in (X, O)
8 }
9 ZOBRIST_TURN = {X: random.getrandbits(64), O: 0}
```

```

10
11 # Mise à jour incrémentale dans apply_move()
12 ns['zhash'] ^= ZOBRIST[(bi, ci, player)]
13 ns['zhash'] ^= ZOBRIST_TURN[player]
14 ns['zhash'] ^= ZOBRIST_TURN[other]

```

Listing 5 – Initialisation et mise à jour incrémentale du hash de Zobrist

5.4 Killer Moves

Lorsqu'un coup provoque une coupure alpha-beta à une certaine profondeur, il est mémorisé comme *killer move* pour cette profondeur. Lors de la prochaine visite d'un nœud à la même profondeur, les killers sont essayés en priorité, avant même le tri général des coups.

L'intuition est que si un coup a été suffisamment bon pour couper la recherche dans une branche, il y a de bonnes chances qu'il soit également fort dans une autre branche au même niveau de l'arbre. On conserve au maximum 2 killers par profondeur pour garder la structure légère.

```

1 kd = killers.setdefault(depth, set())
2 kd.add((bi, ci))
3 if len(kd) > 2:
4     kd.pop()

```

Listing 6 – Mise à jour des killer moves après une coupure

5.5 History Heuristic

Le *history heuristic* maintient un score par triplet (joueur, bi, ci), incrémenté de 2^{depth} à chaque coupure. Les coups historiquement efficaces quelle que soit la profondeur courante sont essayés en priorité.

Complémentaire aux killer moves (qui sont locaux à une profondeur), le history heuristic capture des tendances globales : si jouer dans la case centrale du morpion central a causé de nombreuses coupures à toutes les profondeurs, ce coup sera systématiquement priorisé. Dans l'implémentation actuelle, ce mécanisme est intégré dans le score de `quick_score()` via les `CELL_WEIGHTS` et `BOARD_WEIGHTS`, et les killers sont la couche prioritaire explicite.

5.6 Aspiration Windows

À partir de la troisième itération de l'IDDFS, la recherche ne démarre plus avec la fenêtre complète $[-\infty, +\infty]$, mais avec une fenêtre étroite centrée autour du score de l'itération précédente :

$$[prev_score - W, prev_score + W] \quad \text{avec } W = 50$$

Si la recherche échoue (le score tombe hors de la fenêtre), on relance avec la fenêtre complète. En pratique, la fenêtre est rarement percée : cela permet d'élaguer davantage de nœuds dans le cas courant.

```

1 if depth > MIN_DEPTH:
2     aw_alpha = prev_score - ASPIRATION_WINDOW # 50
3     aw_beta  = prev_score + ASPIRATION_WINDOW
4 else:
5     aw_alpha = -math.inf
6     aw_beta  =  math.inf
7
8 val = alphabeta(ns, depth-1, aw_alpha, aw_beta, ...)
9
10 # Echec de fenetre : on relance avec fenetre complete
11 if val <= aw_alpha or val >= aw_beta:
12     val = alphabeta(ns, depth-1, -math.inf, math.inf, ...)
```

Listing 7 – Aspiration Windows dans la boucle IDDFS

5.7 Late Move Reduction (LMR)

Le *Late Move Reduction* est une heuristique qui réduit la profondeur de recherche pour les coups tardifs dans la liste triée, en supposant qu'ils sont probablement mauvais. Concrètement, si un coup est en position ≥ 3 dans la liste triée et que la profondeur est ≥ 3 , la recherche est d'abord effectuée à profondeur $d - 2$ au lieu de $d - 1$. Si le résultat réduit dépasse alpha (coup possiblement bon), une recherche à profondeur normale $d - 1$ est alors lancée en confirmation.

```

1 LMR_MIN_DEPTH      = 3    # profondeur minimale pour activer LMR
2 LMR_MIN_MOVE_INDEX = 3    # index minimal du coup dans la liste
   triee
3
4 if (move_idx >= LMR_MIN_MOVE_INDEX
5     and depth >= LMR_MIN_DEPTH
6     and not is_terminal(ns)):
7     # Recherche reduite d'abord
8     val = alphabeta(ns, depth-2, alpha, beta, False, ...)
9     # Si le coup semble bon, on refait a profondeur normale
10    if val > alpha:
11        val = alphabeta(ns, depth-1, alpha, beta, False, ...)
```

```

12 else :
13     val = alphabeta(ns, depth-1, alpha, beta, False, ...)

```

Listing 8 – LMR dans le nœud Maximizing

5.8 Move Ordering unifié

Les sources d'information pour trier les coups sont combinées dans la fonction `ordered_moves()` par priorité décroissante :

Priorité	Source	Raison
1	Coup de la TT (itération précédente)	Meilleur coup calculé à profondeur $d + 1$
2	Killer moves	A causé une coupure à cette profondeur
3	<code>quick_score()</code>	Victoire immédiate, blocage, position, redirection

Le tri optimal des coups est la clé de l'efficacité de l'élagage Alpha-Beta : dans le cas idéal (meilleur coup toujours en premier), la complexité est $O(b^{d/2})$ au lieu de $O(b^d)$.

6. Heuristique d'évaluation

6.1 Rôle et enjeux

La fonction `heuristic(st, ai)` est appelée à chaque nœud à la profondeur maximale (ou en cas de timeout). Elle doit approximer la qualité d'une position sans simuler la partie jusqu'au bout. La qualité de l'heuristique détermine directement la force de jeu : une mauvaise heuristique peut rendre inutile même une très grande profondeur de recherche.

6.2 Cinq critères combinés

Critère 1 — Résultat terminal. Si la position est terminale, on retourne $\pm 100\,000$. Ces valeurs absolues garantissent qu'une victoire réelle sera toujours préférée à tout gain heuristique.

Critère 2 — Morpions gagnés pondérés. Chaque mini-morpion remporté vaut $200 \times \text{BOARD_WEIGHTS}[bi]$ points. Le poids reflète le nombre de combinaisons gagnantes auxquelles appartient chaque position :

```

1 BOARD_WEIGHTS = [3, 2, 3, 2, 4, 2, 3, 2, 3] # coins=3, bords=2,
        centre=4

```

```
2 CELL_WEIGHTS = [3, 2, 3, 2, 4, 2, 3, 2, 3]
```

Listing 9 – Poids positionnels des morpions et des cases

Critère 3 — Contrôle des cases individuelles. Dans les morpions non conclus, chaque case occupée rapporte $\text{CELL_WEIGHTS}[ci] \times \text{BOARD_WEIGHTS}[bi]$ points. Le poids double (case $\times$ morpion) valorise davantage les cases centrales des morpions centraux.

Critère 4 — Menaces alignées (macro et mini). Pour chacune des 8 combinaisons gagnantes, on détecte les alignements partiels :

Situation	Score
2 morpions alignés, 1 case libre (macro)	± 800
1 morpion, 2 cases libres (macro)	± 80
2 pions alignés dans un mini-morpion, 1 case libre	$\pm 15 \times \text{BOARD_WEIGHTS}[bi]$

Critère 5 — Qualité de la redirection. Ce critère modélise explicitement la contrainte de redirection, absente de la plupart des heuristiques simples. Si le coup joué envoie l’adversaire dans le morpion central ($\text{BOARD_WEIGHTS} = 4$), c’est défavorable pour nous car il y a plus de pouvoir stratégique. Un bonus/malus de $\pm 20 \times \text{BOARD_WEIGHTS}[\text{active_board}]$ est appliqué selon le joueur dont c’est le tour.

7. Interface et modes de jeu

7.1 Affichage texte

La grille 9×9 est affichée avec des séparateurs visuels entre les macro-blocs. Des codes de couleur ANSI colorisent les pions (X en bleu, O en rouge) et indiquent le plateau actif (cyan si contraint, vert si libre). Une ligne de légende résume les mini-morpions conclus.

7.2 Convention de saisie

Les coups sont saisis au format « colonne ligne » (deux entiers de 1 à 9), conformément au sujet. La conversion interne utilise les formules :

```
1 def coords_to_idx(col, row):
2     c, r = col-1, row-1
3     return (r//3)*3 + (c//3), (r%3)*3 + (c%3)
```

Listing 10 – Conversion coordonnées (col, row) vers indices (bi, ci)

7.3 Mode IA vs IA

Un second mode (`play_ai_vs_ai`) permet de faire s'affronter deux IA sur des machines différentes. L'IA locale calcule et affiche son coup ; l'opérateur le transmet à l'autre machine, puis saisit en retour le coup de l'IA adverse.

8. Analyse des performances

8.1 Gain de l'Iterative Deepening

En début de partie (facteur de branchement ≈ 50 – 70), l'IDDFS avec les optimisations de tri atteint typiquement la profondeur 4–6 dans la limite de 9 secondes. En milieu de partie (≈ 20 coups restants), on atteint 8–12. En fin de partie (< 10 coups), la recherche peut être exhaustive.

Sans IDDFS, une profondeur fixe trop grande provoquerait des timeouts, et une profondeur trop petite gâcherait le temps disponible. L'IDDFS utilise toujours le temps optimal.

8.2 Impact de la transposition table

La TT est particulièrement efficace dans ce jeu car de nombreuses positions se rejoignent par des chemins différents (le même état peut être atteint en jouant les mêmes coups dans un ordre différent). Le stockage du flag `EXACT/LOWER/UPPER` permet de réutiliser des évaluations partielles, ce qui n'est pas possible avec une TT naïve.

8.3 Synergie des optimisations

Les sept optimisations se renforcent mutuellement :

- l'IDDFS fournit des résultats à chaque profondeur qui améliorent le move ordering de l'itération suivante ;
- le meilleur move ordering améliore l'élagage Alpha-Beta ;
- de meilleures coupures Alpha-Beta enrichissent les killers et le history heuristic ;
- les Aspiration Windows réduisent encore les nœuds évalués dans les itérations profondes ;
- le LMR élague les coups tardifs probablement mauvais sans perdre en qualité grâce à la re-recherche sur les faux positifs.

8.4 Limites et pistes d'amélioration

- L'heuristique ne modélise pas toutes les subtilités de la redirection (forcer l'adversaire dans un morpion presque plein peut être très fort).

- Une *recherche en quiescence* permettrait d'éviter l'effet d'horizon : ne pas arrêter la recherche en plein milieu d'une séquence de capture de morpions.
- Le *Principal Variation Search* (PVS / NegaScout) raffinerait encore l'élagage en cherchant d'abord la variation principale avec une fenêtre nulle.

9. Conclusion

Notre IA repose sur une base Minimax Alpha-Beta enrichie de sept optimisations complémentaires :

- **Iterative Deepening** pour l'adaptation automatique au temps ;
- **Transposition Table avec flags** pour éviter les recalculs ;
- **Hash de Zobrist** (avec bit de tour) pour un accès $O(1)$ à la TT ;
- **Killer Moves** pour un move ordering local par profondeur ;
- **History Heuristic** capturée via les poids positionnels et les killers ;
- **Aspiration Windows** pour réduire les nœuds évalués dans les itérations profondes ;
- **Late Move Reduction** pour élaguer les coups tardifs sans perte de qualité.

Le résultat est une IA qui exploite intégralement le budget de 10 secondes, adapte sa profondeur à la phase de jeu, et produit des coups de qualité croissante au fur et à mesure des itérations. L'heuristique enrichie, qui modélise explicitement la contrainte de redirection, lui confère une dimension stratégique que les heuristiques naïves ignorent.